

CHAPTER 4 NODE JS



Chapter 4: Microservices Architecture & Enterprise Scalability

Welcome to the **expert level**! In this chapter, we'll transform your monolithic backend into a **scalable microservices architecture** that can handle millions of users.

Chapter 4 Learning Goals

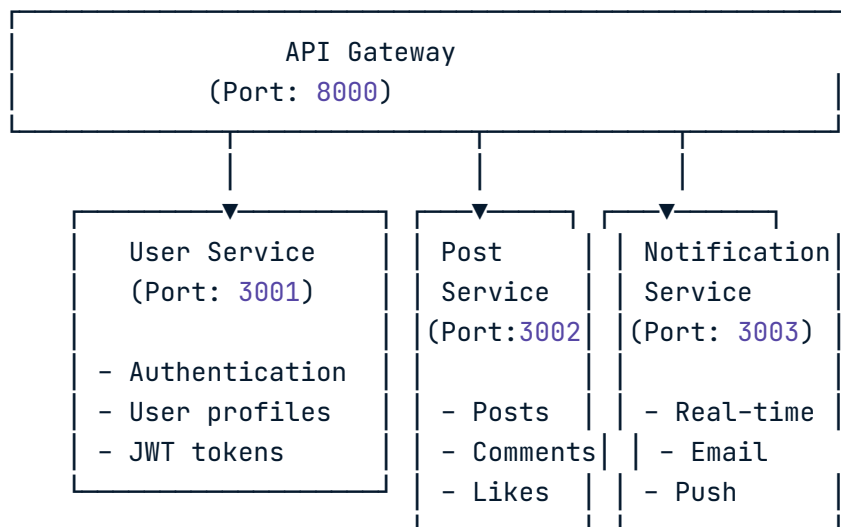
By the end of this chapter, you'll master:

- **Microservices Architecture** design and implementation
- **API Gateway** with routing and load balancing
- **Message Queues** for asynchronous processing
- **Event-Driven Communication** between services
- **Database Per Service** patterns
- **Service Discovery** and health monitoring
- **Distributed Tracing** and logging
- **Container Orchestration** with Docker & Kubernetes

Microservices Architecture Design

1. Breaking Down the Monolith

Let's restructure our application into focused microservices:



 Copy

2. Service Structure Template

```
// services/user-service/
```

 Copy

```
├── src/
│   ├── controllers/
│   │   └── user.controller.js
│   ├── models/
│   │   └── User.model.js
│   ├── routes/
│   │   └── user.routes.js
│   ├── middleware/
│   │   └── auth.middleware.js
│   ├── services/
│   │   └── user.service.js
│   ├── events/
│   │   ├── publishers/
│   │   └── subscribers/
│   └── app.js
├── config/
│   ├── database.js
│   └── environment.js
├── tests/
├── Dockerfile
├── package.json
└── server.js
```

API Gateway Implementation

1. Setup API Gateway with Express

```
mkdir api-gateway
cd api-gateway
npm init -y
npm install express http-proxy-middleware cors helmet morgan winston
```

 Copy

```
// api-gateway/src/app.js
const express = require('express');
const { createProxyMiddleware } = require('http-proxy-middleware');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');
const logger = require('./utils/logger');
```

 Copy

```
const app = express();

// Security middleware
app.use(helmet());
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || ['http://localhost:3000'],
  credentials: true
}));

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP'
});
app.use('/api/', limiter);

// Logging
app.use(morgan('combined', {
  stream: { write: message => logger.info(message.trim()) }
}));

// Body parsing
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// Service discovery configuration
const services = {
  user: {
    target: process.env.USER_SERVICE_URL || 'http://localhost:3001',
    changeOrigin: true,
    timeout: 10000,
    retries: 3
  },
  post: {
    target: process.env.POST_SERVICE_URL || 'http://localhost:3002',
    changeOrigin: true,
    timeout: 10000,
    retries: 3
  },
  notification: {
    target: process.env.NOTIFICATION_SERVICE_URL || 'http://localhost:3003',
    changeOrigin: true,
    timeout: 10000,
    retries: 3
  }
};

// Health check middleware
```

```
const healthCheck = (serviceName) => {
  return async (req, res, next) => {
    try {
      const service = services[serviceName];
      const healthUrl = `${service.target}/health`;

      // Simple health check (in production, use more sophisticated methods)
      const fetch = require('node-fetch');
      const response = await fetch(healthUrl, { timeout: 5000 });

      if (response.ok) {
        next();
      } else {
        throw new Error(`Service ${serviceName} is unhealthy`);
      }
    } catch (error) {
      logger.error(`Health check failed for ${serviceName}:`, error);
      res.status(503).json({
        success: false,
        message: `Service ${serviceName} is currently unavailable`,
        error: 'SERVICE_UNAVAILABLE'
      });
    }
  };
};

// Load balancing function (simple round-robin)
class LoadBalancer {
  constructor() {
    this.counters = {};
  }

  getTarget(serviceName, instances) {
    if (!this.counters[serviceName]) {
      this.counters[serviceName] = 0;
    }

    const index = this.counters[serviceName] % instances.length;
    this.counters[serviceName]++;

    return instances[index];
  }
}

const loadBalancer = new LoadBalancer();

// Proxy middleware with load balancing
const createServiceProxy = (serviceName, pathRewrite = {}) => {
  return createProxyMiddleware({
```

```

    target: services[serviceName].target,
    changeOrigin: true,
    pathRewrite,
    timeout: services[serviceName].timeout,
    retries: services[serviceName].retries,
    onError: (err, req, res) => {
      logger.error(`Proxy error for ${serviceName}:`, err);
      res.status(503).json({
        success: false,
        message: `Service ${serviceName} is currently unavailable`,
        error: 'PROXY_ERROR'
      });
    },
    onProxyReq: (proxyReq, req, res) => {
      // Add correlation ID for distributed tracing
      const correlationId = req.headers['x-correlation-id'] ||
        require('uuid').v4();
      proxyReq.setHeader('x-correlation-id', correlationId);
      proxyReq.setHeader('x-forwarded-for', req.ip);

      logger.info(`Proxying ${req.method} ${req.url} to ${serviceName}`, {
        correlationId,
        userAgent: req.headers['user-agent']
      });
    }
  });
};

// Route definitions with health checks
app.use('/api/v1/auth',
  healthCheck('user'),
  createServiceProxy('user', { '^/api/v1/auth': '/api/v1/auth' })
);

app.use('/api/v1/users',
  healthCheck('user'),
  createServiceProxy('user', { '^/api/v1/users': '/api/v1/users' })
);

app.use('/api/v1/posts',
  healthCheck('post'),
  createServiceProxy('post', { '^/api/v1/posts': '/api/v1/posts' })
);

app.use('/api/v1/notifications',
  healthCheck('notification'),
  createServiceProxy('notification', { '^/api/v1/notifications':
    '/api/v1/notifications' })
);

```

```
// Gateway health check
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'OK',
    timestamp: new Date().toISOString(),
    services: Object.keys(services),
    uptime: process.uptime()
  });
});

// Catch-all for undefined routes
app.all('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: 'Route not found'
  });
});

// Global error handler
app.use((err, req, res, next) => {
  logger.error('Gateway error:', err);
  res.status(500).json({
    success: false,
    message: 'Internal gateway error'
  });
});

module.exports = app;
```

2. Service Registry & Discovery

```
// api-gateway/src/services/service-registry.js
const EventEmitter = require('events');
const fetch = require('node-fetch');
const logger = require('../utils/logger');

class ServiceRegistry extends EventEmitter {
  constructor() {
    super();
    this.services = new Map();
    this.healthCheckInterval = 30000; // 30 seconds
    this.startHealthChecks();
  }

  // Register a service
  register(name, config) {
```

 Copy

```
const service = {
  name,
  ...config,
  lastHealthCheck: null,
  healthy: true,
  instances: config.instances || [config.target]
};

this.services.set(name, service);
this.emit('serviceRegistered', service);
logger.info(`Service registered: ${name}`);
}

// Unregister a service
unregister(name) {
  const service = this.services.get(name);
  if (service) {
    this.services.delete(name);
    this.emit('serviceUnregistered', service);
    logger.info(`Service unregistered: ${name}`);
  }
}

// Get service configuration
getService(name) {
  return this.services.get(name);
}

// Get healthy instances of a service
getHealthyInstances(name) {
  const service = this.services.get(name);
  return service?.healthy ? service.instances : [];
}

// Health check for all services
async performHealthChecks() {
  const promises = Array.from(this.services.keys()).map(async (serviceName) => {
    const service = this.services.get(serviceName);

    try {
      const response = await fetch(`${service.target}/health`, {
        timeout: 5000,
        method: 'GET'
      });

      const isHealthy = response.ok;
      const wasHealthy = service.healthy;

      service.healthy = isHealthy;
    }
  });
}
```

```

    service.lastHealthCheck = new Date();

    if (isHealthy !== wasHealthy) {
      this.emit('healthStatusChanged', {
        service: serviceName,
        healthy: isHealthy,
        timestamp: new Date()
      });

      logger.info(`Service ${serviceName} health changed: ${isHealthy ?
'healthy' : 'unhealthy'}`);
    }

    } catch (error) {
      const wasHealthy = service.healthy;
      service.healthy = false;
      service.lastHealthCheck = new Date();

      if (wasHealthy) {
        this.emit('healthStatusChanged', {
          service: serviceName,
          healthy: false,
          error: error.message,
          timestamp: new Date()
        });

        logger.error(`Service ${serviceName} health check failed:`, error);
      }
    }
  });

  await Promise.allSettled(promises);
}

// Start periodic health checks
startHealthChecks() {
  setInterval(() => {
    this.performHealthChecks();
  }, this.healthCheckInterval);

  // Initial health check
  setTimeout(() => this.performHealthChecks(), 1000);
}

// Get registry status
getStatus() {
  const services = Array.from(this.services.entries()).map(([name, service]) =>
({
    name,

```

```

    healthy: service.healthy,
    lastHealthCheck: service.lastHealthCheck,
    target: service.target,
    instances: service.instances.length
  }));

  return {
    totalServices: services.length,
    healthyServices: services.filter(s => s.healthy).length,
    services
  };
}
}

module.exports = new ServiceRegistry();

```

Message Queue System

1. Setup Redis-based Message Queue

```
npm install bull ioredis
```

 Copy

```

// shared/message-queue/queue-manager.js
const Bull = require('bull');
const Redis = require('ioredis');
const logger = require('../utils/logger');

class QueueManager {
  constructor() {
    this.redis = new Redis({
      host: process.env.REDIS_HOST || 'localhost',
      port: process.env.REDIS_PORT || 6379,
      password: process.env.REDIS_PASSWORD,
      retryDelayOnFailover: 100,
      maxRetriesPerRequest: 3
    });
  }

  this.queues = new Map();
  this.setupQueues();
}

setupQueues() {
  // Email queue

```

 Copy

```
this.createQueue('email', {
  defaultJobOptions: {
    attempts: 3,
    backoff: {
      type: 'exponential',
      delay: 2000
    },
    removeOnComplete: 100,
    removeOnFail: 50
  }
});

// Notification queue
this.createQueue('notifications', {
  defaultJobOptions: {
    attempts: 5,
    backoff: 'exponential',
    removeOnComplete: 50,
    removeOnFail: 50
  }
});

// File processing queue
this.createQueue('file-processing', {
  defaultJobOptions: {
    attempts: 2,
    timeout: 60000, // 1 minute timeout
    removeOnComplete: 20,
    removeOnFail: 20
  }
});

// Analytics queue
this.createQueue('analytics', {
  defaultJobOptions: {
    attempts: 1,
    removeOnComplete: 1000,
    removeOnFail: 100
  }
});

}

createQueue(name, options = {}) {
  const queue = new Bull(name, {
    redis: {
      host: process.env.REDIS_HOST || 'localhost',
      port: process.env.REDIS_PORT || 6379,
      password: process.env.REDIS_PASSWORD
    },
  },
```

```

    ...options
  });

  // Queue event listeners
  queue.on('completed', (job, result) => {
    logger.info(`Job completed in queue ${name}:`, {
      jobId: job.id,
      jobType: job.name,
      processingTime: job.processedOn - job.processedOn,
      result: typeof result === 'string' ? result.substring(0, 100) :
'completed'
    });
  });

  queue.on('failed', (job, err) => {
    logger.error(`Job failed in queue ${name}:`, {
      jobId: job.id,
      jobType: job.name,
      error: err.message,
      attempts: job.attemptsMade,
      maxAttempts: job.opts.attempts
    });
  });

  queue.on('stalled', (job) => {
    logger.warn(`Job stalled in queue ${name}:`, {
      jobId: job.id,
      jobType: job.name
    });
  });

  this.queues.set(name, queue);
  return queue;
}

// Add job to queue
async addJob(queueName, jobName, data, options = {}) {
  const queue = this.queues.get(queueName);
  if (!queue) {
    throw new Error(`Queue ${queueName} not found`);
  }

  const job = await queue.add(jobName, data, {
    priority: options.priority || 0,
    delay: options.delay || 0,
    ...options
  });

  logger.info(`Job added to queue ${queueName}:`, {

```

```
        jobId: job.id,
        jobName,
        data: typeof data === 'object' ? Object.keys(data) : 'primitive'
    });

    return job;
}

// Get queue instance
getQueue(name) {
    return this.queues.get(name);
}

// Process jobs in queue
process(queueName, processor, concurrency = 1) {
    const queue = this.queues.get(queueName);
    if (!queue) {
        throw new Error(`Queue ${queueName} not found`);
    }

    queue.process(concurrency, async (job) => {
        const startTime = Date.now();

        try {
            const result = await processor(job.data, job);

            logger.info(`Job processed successfully:`, {
                queueName,
                jobId: job.id,
                processingTime: Date.now() - startTime
            });

            return result;
        } catch (error) {
            logger.error(`Job processing failed:`, {
                queueName,
                jobId: job.id,
                error: error.message,
                processingTime: Date.now() - startTime
            });

            throw error;
        }
    });
}

// Get queue statistics
async getQueueStats(queueName) {
    const queue = this.queues.get(queueName);
```

```
if (!queue) {
  return null;
}

const [waiting, active, completed, failed, delayed, paused] = await
Promise.all([
  queue.getWaiting(),
  queue.getActive(),
  queue.getCompleted(),
  queue.getFailed(),
  queue.getDelayed(),
  queue.getPaused()
]);

return {
  queueName,
  counts: {
    waiting: waiting.length,
    active: active.length,
    completed: completed.length,
    failed: failed.length,
    delayed: delayed.length,
    paused: paused.length
  },
  jobs: {
    waiting: waiting.slice(0, 5).map(job => ({
      id: job.id,
      name: job.name,
      createdAt: job.timestamp
    })),
    active: active.slice(0, 5).map(job => ({
      id: job.id,
      name: job.name,
      progress: job.progress(),
      createdAt: job.timestamp
    }))
  }
};
}

// Get all queues statistics
async getAllStats() {
  const stats = {};

  for (const [name, queue] of this.queues) {
    stats[name] = await this.getQueueStats(name);
  }

  return stats;
}
```

```

    }

    // Cleanup method
    async cleanup() {
      const promises = Array.from(this.queues.values()).map(queue => queue.close());
      await Promise.all(promises);
      await this.redis.quit();
    }
  }

  module.exports = new QueueManager();

```

2. Job Processors

```

// shared/message-queue/processors/email-processor.js
const nodemailer = require('nodemailer');
const logger = require('../../utils/logger');

class EmailProcessor {
  constructor() {
    this.transporter = nodemailer.createTransporter({
      host: process.env.SMTP_HOST,
      port: process.env.SMTP_PORT,
      secure: false,
      auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS
      }
    });
  }

  async process(data, job) {
    const { to, subject, html, text, template, templateData } = data;

    try {
      // Update job progress
      await job.progress(10);

      let emailHtml = html;
      let emailText = text;

      // Process template if provided
      if (template && templateData) {
        emailHtml = await this.renderTemplate(template, templateData);
        await job.progress(30);
      }
    }
  }
}

```

 Copy

```
// Prepare mail options
const mailOptions = {
  from: ` "${process.env.APP_NAME}" <${process.env.SMTP_USER}>`,
  to,
  subject,
  html: emailHtml,
  text: emailText
};

await job.progress(50);

// Send email
const info = await this.transporter.sendMail(mailOptions);

await job.progress(100);

logger.info('Email sent successfully:', {
  to,
  subject,
  messageId: info.messageId
});

return {
  success: true,
  messageId: info.messageId,
  to,
  subject
};
} catch (error) {
  logger.error('Email sending failed:', {
    to,
    subject,
    error: error.message
  });

  throw error;
}

}

async renderTemplate(templateName, data) {
  // Simple template rendering (use handlebars or ejs for more complex
  templates)
  const templates = {
    welcome: `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
      auto;">
        <h2>Welcome to MyApp, ${name}</h2>
        <p>Thank you for joining our community.</p>
    `
  }
}
```

```

    <p>Your account has been created successfully.</p>
  </div>
  ,
  passwordReset: `
    <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
auto;">
      <h2>Password Reset Request</h2>
      <p>Hi {{name}},</p>
      <p>Click the link below to reset your password:</p>
      <a href="{{resetUrl}}" style="background: #007bff; color: white;
padding: 10px 20px; text-decoration: none; border-radius: 5px;">
        Reset Password
      </a>
      <p>This link expires in 10 minutes.</p>
    </div>
  ,
};

let template = templates[templateName];
if (!template) {
  throw new Error(`Template ${templateName} not found`);
}

// Simple replacement (use proper template engine in production)
for (const [key, value] of Object.entries(data)) {
  template = template.replace(new RegExp(`{{${key}}}`, 'g'), value);
}

return template;
}
}

module.exports = EmailProcessor;

```

```

// shared/message-queue/processors/notification-processor.js
const logger = require('../utils/logger');

class NotificationProcessor {
  async process(data, job) {
    const { type, userId, message, payload } = data;

    try {
      await job.progress(20);

      switch (type) {
        case 'push':
          await this.sendPushNotification(userId, message, payload);

```

 Copy

```
        break;

    case 'sms':
        await this.sendSMSNotification(userId, message, payload);
        break;

    case 'in-app':
        await this.saveInAppNotification(userId, message, payload);
        break;

    case 'webhook':
        await this.sendWebhookNotification(payload);
        break;

    default:
        throw new Error(`Unknown notification type: ${type}`);
}

await job.progress(100);

return {
    success: true,
    type,
    userId,
    processedAt: new Date()
};

} catch (error) {
    logger.error('Notification processing failed:', {
        type,
        userId,
        error: error.message
    });

    throw error;
}

}

async sendPushNotification(userId, message, payload) {
    // Implement push notification logic
    // This could integrate with Firebase Cloud Messaging, Apple Push
    Notifications, etc.
    logger.info(`Sending push notification to user ${userId}: ${message}`);

    // Simulate processing time
    await new Promise(resolve => setTimeout(resolve, 500));
}

async sendSMSNotification(userId, message, payload) {
```

```
// Implement SMS logic (Twilio, AWS SNS, etc.)
logger.info(`Sending SMS to user ${userId}: ${message}`);

await new Promise(resolve => setTimeout(resolve, 300));
}

async saveInAppNotification(userId, message, payload) {
  // Save notification to database for in-app display
  const Notification = require('../models/Notification.model');

  await Notification.create({
    recipient: userId,
    message,
    type: 'system',
    data: payload
  });

  logger.info(`Saved in-app notification for user ${userId}`);
}

async sendWebhookNotification(payload) {
  // Send webhook to external services
  const fetch = require('node-fetch');

  if (payload.webhookUrl) {
    await fetch(payload.webhookUrl, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload.data)
    });
  }
}

module.exports = NotificationProcessor;
```

3. Queue Integration in Services

```
// services/user-service/src/events/publishers/user-events.js  Copy
const queueManager = require('../../../../shared/message-queue/queue-manager');

class UserEventPublisher {

  // Publish user registered event
  static async publishUserRegistered(userData) {
    // Send welcome email
    await queueManager.addJob('email', 'welcome-email', {
```

```
    to: userData.email,
    template: 'welcome',
    templateData: {
      name: userData.name
    }
  });

// Send welcome notification
await queueManager.addJob('notifications', 'welcome-notification', {
  type: 'in-app',
  userId: userData.id,
  message: 'Welcome to MyApp! Your journey starts here.',
  payload: { type: 'welcome' }
});

// Analytics tracking
await queueManager.addJob('analytics', 'user-registered', {
  userId: userData.id,
  email: userData.email,
  timestamp: new Date(),
  source: 'web'
});
}

// Publish password reset event
static async publishPasswordResetRequested(userData, resetToken) {
  await queueManager.addJob('email', 'password-reset-email', {
    to: userData.email,
    template: 'passwordReset',
    templateData: {
      name: userData.name,
      resetUrl: `${process.env.CLIENT_URL}/reset-password/${resetToken}`
    }
  }, {
    priority: 5 // Higher priority for security-related emails
  });
}

// Publish user updated event
static async publishUserUpdated(userData, changes) {
  // Analytics for profile updates
  await queueManager.addJob('analytics', 'user-updated', {
    userId: userData.id,
    changes: Object.keys(changes),
    timestamp: new Date()
  });

  // Notify other services about user update
  await queueManager.addJob('notifications', 'user-profile-updated', {
```

```

    type: 'webhook',
    payload: {
      webhookUrl: process.env.USER_UPDATE_WEBHOOK,
      data: {
        userId: userData.id,
        changes,
        timestamp: new Date()
      }
    }
  });
}
}

module.exports = UserEventPublisher;

```

Event-Driven Architecture

1. Event Bus Implementation

```

// shared/events/event-bus.js
const EventEmitter = require('events');
const Redis = require('ioredis');
const logger = require('../utils/logger');

class EventBus extends EventEmitter {
  constructor() {
    super();
    this.publisher = new Redis({
      host: process.env.REDIS_HOST || 'localhost',
      port: process.env.REDIS_PORT || 6379,
      password: process.env.REDIS_PASSWORD
    });

    this.subscriber = new Redis({
      host: process.env.REDIS_HOST || 'localhost',
      port: process.env.REDIS_PORT || 6379,
      password: process.env.REDIS_PASSWORD
    });

    this.setupSubscriber();
  }

  setupSubscriber() {
    this.subscriber.on('message', (channel, message) => {

```

 Copy

```
try {
  const event = JSON.parse(message);
  this.emit(channel, event);

  logger.info('Event received:', {
    channel,
    eventId: event.id,
    type: event.type,
    timestamp: event.timestamp
  });
} catch (error) {
  logger.error('Failed to parse event message:', error);
}
});

this.subscriber.on('error', (error) => {
  logger.error('Event subscriber error:', error);
});
}

// Publish event
async publish(channel, eventData) {
  const event = {
    id: require('uuid').v4(),
    timestamp: new Date().toISOString(),
    source: process.env.SERVICE_NAME || 'unknown',
    ...eventData
  };

  try {
    await this.publisher.publish(channel, JSON.stringify(event));

    logger.info('Event published:', {
      channel,
      eventId: event.id,
      type: event.type
    });

    return event.id;
  } catch (error) {
    logger.error('Failed to publish event:', error);
    throw error;
  }
}

// Subscribe to channel
async subscribe(channel) {
  try {
    await this.subscriber.subscribe(channel);
```

```

    logger.info(`Subscribed to channel: ${channel}`);
  } catch (error) {
    logger.error(`Failed to subscribe to channel ${channel}:`, error);
    throw error;
  }
}

// Unsubscribe from channel
async unsubscribe(channel) {
  try {
    await this.subscriber.unsubscribe(channel);
    logger.info(`Unsubscribed from channel: ${channel}`);
  } catch (error) {
    logger.error(`Failed to unsubscribe from channel ${channel}:`, error);
    throw error;
  }
}

// Pattern subscription
async psubscribe(pattern) {
  try {
    await this.subscriber.psubscribe(pattern);
    logger.info(`Pattern subscribed: ${pattern}`);
  } catch (error) {
    logger.error(`Failed to pattern subscribe ${pattern}:`, error);
    throw error;
  }
}
}

module.exports = new EventBus();

```

2. Event Definitions

```

// shared/events/event-types.js
const EVENT_TYPES = {
  // User events
  USER_REGISTERED: 'user.registered',
  USER_UPDATED: 'user.updated',
  USER_DELETED: 'user.deleted',
  USER_LOGIN: 'user.login',

  // Post events
  POST_CREATED: 'post.created',
  POST_UPDATED: 'post.updated',
  POST_DELETED: 'post.deleted',
  POST_LIKED: 'post.liked',

```

 Copy

```

POST_UNLIKED: 'post.unliked',

// Comment events
COMMENT_CREATED: 'comment.created',
COMMENT_UPDATED: 'comment.updated',
COMMENT_DELETED: 'comment.deleted',

// Notification events
NOTIFICATION_CREATED: 'notification.created',
NOTIFICATION_READ: 'notification.read',

// System events
SYSTEM_ERROR: 'system.error',
SYSTEM_MAINTENANCE: 'system.maintenance'
};

module.exports = EVENT_TYPES;

```

3. Service Event Handlers

```

// services/notification-service/src/events/handlers/user-
events.handler.js
const eventBus = require('../../../../shared/events/event-bus');
const EVENT_TYPES = require('../../../../shared/events/event-types');
const NotificationService = require('../../../../services/notification.service');
const logger = require('../../../../shared/utils/logger');

class UserEventsHandler {

  static async initialize() {
    // Subscribe to user events
    await eventBus.subscribe('user.*');

    // Handle user registered event
    eventBus.on(EVENT_TYPES.USER_REGISTERED, async (event) => {
      try {
        await this.handleUserRegistered(event);
      } catch (error) {
        logger.error('Failed to handle user registered event:', error);
      }
    });

    // Handle user updated event
    eventBus.on(EVENT_TYPES.USER_UPDATED, async (event) => {
      try {
        await this.handleUserUpdated(event);
      } catch (error) {

```

 Copy

```

        logger.error('Failed to handle user updated event:', error);
    }
});
}

static async handleUserRegistered(event) {
    const { userId, email, name } = event.data;

    // Create welcome notification
    await NotificationService.createNotification({
        recipient: userId,
        type: 'welcome',
        message: `Welcome to MyApp, ${name}!`,
        data: {
            onboardingStep: 1,
            completedAt: new Date()
        }
    });

    logger.info(`Welcome notification created for user: ${userId}`);
}

static async handleUserUpdated(event) {
    const { userId, changes } = event.data;

    // If profile picture was updated, notify followers
    if (changes.includes('avatar')) {
        await NotificationService.notifyFollowers(userId, {
            type: 'profile_update',
            message: 'updated their profile picture',
            data: { updateType: 'avatar' }
        });
    }

    logger.info(`Profile update notifications sent for user: ${userId}`);
}
}

module.exports = UserEventsHandler;

```

Database Per Service Pattern

1. User Service Database Schema



```
// services/user-service/src/models/User.model.js
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    maxlength: [50, 'Name cannot exceed 50 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Invalid email']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters'],
    select: false
  },
  role: {
    type: String,
    enum: ['user', 'admin', 'moderator'],
    default: 'user'
  },
  profile: {
    avatar: {
      type: String,
      default: null
    },
    bio: {
      type: String,
      maxlength: [500, 'Bio cannot exceed 500 characters']
    },
    website: {
      type: String,
      validate: {
        validator: function(v) {
          return !v || /^https?:\/\/\w+\.test(v);
        },
        message: 'Invalid website URL'
      }
    }
  }
});
```

```
    location: String,
    dateOfBirth: Date
  },
  preferences: {
    emailNotifications: {
      type: Boolean,
      default: true
    },
    pushNotifications: {
      type: Boolean,
      default: true
    },
    privacy: {
      profileVisibility: {
        type: String,
        enum: ['public', 'private', 'friends'],
        default: 'public'
      }
    }
  },
  verification: {
    email: {
      verified: {
        type: Boolean,
        default: false
      },
      token: String,
      tokenExpires: Date
    }
  },
  security: {
    twoFactorEnabled: {
      type: Boolean,
      default: false
    },
    lastLogin: Date,
    loginAttempts: {
      type: Number,
      default: 0
    },
    lockUntil: Date
  },
  social: {
    followersCount: {
      type: Number,
      default: 0
    },
    followingCount: {
      type: Number,
```

```
        default: 0
      },
      postsCount: {
        type: Number,
        default: 0
      }
    },
    status: {
      type: String,
      enum: ['active', 'inactive', 'suspended', 'deleted'],
      default: 'active'
    }
  }, {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  });

// Indexes for performance
userSchema.index({ email: 1 });
userSchema.index({ 'verification.email.token': 1 });
userSchema.index({ status: 1 });
userSchema.index({ createdAt: -1 });

// Virtual fields
userSchema.virtual('isLocked').get(function() {
  return !(this.security.lockUntil && this.security.lockUntil > Date.now());
});

// Pre-save middleware
userSchema.pre('save', async function(next) {
  // Hash password if modified
  if (this.isModified('password')) {
    const salt = await bcrypt.genSalt(12);
    this.password = await bcrypt.hash(this.password, salt);
  }
  next();
});

// Instance methods
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

userSchema.methods.getSignedJwtToken = function() {
  return jwt.sign(
    {
      id: this._id,
      email: this.email,
```

```

    role: this.role
  },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRE || '7d' }
);
};

// Static methods
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() });
};

module.exports = mongoose.model('User', userSchema);

```

2. Post Service Database Schema

```

// services/post-service/src/models/Post.model.js
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  // Author reference (stored as ID only, no population across services)
  authorId: {
    type: String, // Store as string to avoid cross-service ObjectId references
    required: true,
    index: true
  },
  // Cache author info to avoid cross-service calls
  authorInfo: {
    name: String,
    avatar: String,
    verified: Boolean
  },
  content: {
    title: {
      type: String,
      required: [true, 'Title is required'],
      trim: true,
      maxLength: [200, 'Title cannot exceed 200 characters']
    },
    body: {
      type: String,
      required: [true, 'Post body is required'],
      maxLength: [5000, 'Post cannot exceed 5000 characters']
    },
    excerpt: {
      type: String,
      maxLength: [300, 'Excerpt cannot exceed 300 characters']
    }
  }
});

```

 Copy

```
    },
    tags: [{
      type: String,
      lowercase: true,
      trim: true,
      maxLength: [30, 'Tag cannot exceed 30 characters']
    }],
    category: {
      type: String,
      required: true,
      enum: ['technology', 'lifestyle', 'business', 'health', 'entertainment',
'other']
    }
  },
  media: {
    images: [{
      url: String,
      alt: String,
      caption: String
    }],
    featuredImage: {
      url: String,
      alt: String
    }
  },
  engagement: {
    likes: [{
      userId: String,
      createdAt: {
        type: Date,
        default: Date.now
      }
    }],
    likesCount: {
      type: Number,
      default: 0
    },
    commentsCount: {
      type: Number,
      default: 0
    },
    viewsCount: {
      type: Number,
      default: 0
    },
    sharesCount: {
      type: Number,
      default: 0
    }
  }
}
```

```

    },
    metadata: {
      readTime: {
        type: Number, // in minutes
        default: 1
      },
      status: {
        type: String,
        enum: ['draft', 'published', 'archived', 'deleted'],
        default: 'draft'
      },
      visibility: {
        type: String,
        enum: ['public', 'private', 'unlisted'],
        default: 'public'
      },
      publishedAt: Date,
      scheduledFor: Date
    },
    seo: {
      slug: {
        type: String,
        unique: true,
        sparse: true // Allow null values but ensure uniqueness when present
      },
      metaDescription: {
        type: String,
        maxLength: [160, 'Meta description cannot exceed 160 characters']
      },
      keywords: [String]
    }
  }, {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  });

// Indexes for performance
postSchema.index({ authorId: 1, createdAt: -1 });
postSchema.index({ 'metadata.status': 1, 'metadata.publishedAt': -1 });
postSchema.index({ 'content.tags': 1 });
postSchema.index({ 'content.category': 1 });
postSchema.index({ 'seo.slug': 1 });
postSchema.index({ 'engagement.likesCount': -1 });

// Text index for search
postSchema.index({
  'content.title': 'text',
  'content.body': 'text',

```

```
'content.tags': 'text'
});

// Virtual fields
postSchema.virtual('url').get(function() {
  return `/posts/${this.seo.slug || this._id}`;
});

postSchema.virtual('isPublished').get(function() {
  return this.metadata.status === 'published' &&
    this.metadata.publishedAt <= new Date();
});

// Pre-save middleware
postSchema.pre('save', function(next) {
  // Generate slug if not exists
  if (!this.seo.slug && this.content.title) {
    this.seo.slug = this.content.title
      .toLowerCase()
      .replace(/^[a-z0-9]/g, '-')
      .replace(/-+/g, '-')
      .replace(/^-|-$/g, '')
      .substring(0, 50);
  }

  // Calculate read time
  if (this.isModified('content.body')) {
    const wordsPerMinute = 200;
    const wordCount = this.content.body.split(/\s+/).length;
    this.metadata.readTime = Math.ceil(wordCount / wordsPerMinute) || 1;
  }

  // Set published date
  if (this.isModified('metadata.status') &&
    this.metadata.status === 'published' &&
    !this.metadata.publishedAt) {
    this.metadata.publishedAt = new Date();
  }

  next();
});

// Instance methods
postSchema.methods.addLike = function(userId) {
  const existingLike = this.engagement.likes.find(
    like => like.userId === userId
  );

  if (!existingLike) {
```

```
this.engagement.likes.push({ userId });
this.engagement.likesCount = this.engagement.likes.length;
return true;
}

return false;
};

postSchema.methods.removeLike = function(userId) {
  const initialLength = this.engagement.likes.length;
  this.engagement.likes = this.engagement.likes.filter(
    like => like.userId !== userId
  );

  if (this.engagement.likes.length < initialLength) {
    this.engagement.likesCount = this.engagement.likes.length;
    return true;
  }

  return false;
};

module.exports = mongoose.model('Post', postSchema);
```



Service Monitoring & Health Checks

1. Advanced Health Check System

```
// shared/monitoring/health-checker.js
const mongoose = require('mongoose');
const Redis = require('ioredis');
const fetch = require('node-fetch');
const logger = require('../utils/logger');

class HealthChecker {
  constructor(serviceName) {
    this.serviceName = serviceName;
    this.checks = new Map();
    this.setupDefaultChecks();
  }

  setupDefaultChecks() {
    // Database connectivity check
    this.addCheck('database', async () => {
```

[Copy](#)

```

const state = mongoose.connection.readyState;
if (state !== 1) {
  throw new Error(`Database not connected. State: ${state}`);
}

// Test database operation
await mongoose.connection.db.admin().ping();

return {
  status: 'healthy',
  details: {
    state: ['disconnected', 'connected', 'connecting', 'disconnecting']
[state],
    host: mongoose.connection.host,
    name: mongoose.connection.name
  }
};
});

// Memory usage check
this.addCheck('memory', async () => {
  const memUsage = process.memoryUsage();
  const heapUsedMB = Math.round(memUsage.heapUsed / 1024 / 1024);
  const heapTotalMB = Math.round(memUsage.heapTotal / 1024 / 1024);
  const usagePercent = Math.round((heapUsedMB / heapTotalMB) * 100);

  if (usagePercent > 90) {
    throw new Error(`High memory usage: ${usagePercent}%`);
  }

  return {
    status: usagePercent > 75 ? 'warning' : 'healthy',
    details: {
      heapUsed: `${heapUsedMB} MB`,
      heapTotal: `${heapTotalMB} MB`,
      usagePercent: `${usagePercent}%`,
      rss: `${Math.round(memUsage.rss / 1024 / 1024)} MB`
    }
  };
});

// Disk space check (if applicable)
this.addCheck('disk', async () => {
  const fs = require('fs');
  const path = require('path');

  try {
    const stats = await fs.promises.statSync(path.resolve('./'));
    return {

```

```

        status: 'healthy',
        details: {
            available: 'Unknown', // Would need platform-specific implementation
            path: path.resolve('./')
        }
    };
} catch (error) {
    throw new Error(`Disk check failed: ${error.message}`);
}
});
}

// Add custom health check
addCheck(name, checkFunction, options = {}) {
    this.checks.set(name, {
        name,
        check: checkFunction,
        timeout: options.timeout || 5000,
        critical: options.critical !== false // Default to critical
    });
}

// Add Redis health check
addRedisCheck(redisInstance, name = 'redis') {
    this.addCheck(name, async () => {
        if (!redisInstance || redisInstance.status !== 'ready') {
            throw new Error('Redis not connected');
        }

        const start = Date.now();
        await redisInstance.ping();
        const latency = Date.now() - start;

        if (latency > 1000) {
            throw new Error(`High Redis latency: ${latency}ms`);
        }

        return {
            status: latency > 500 ? 'warning' : 'healthy',
            details: {
                latency: `${latency}ms`,
                status: redisInstance.status,
                host: redisInstance.options.host,
                port: redisInstance.options.port
            }
        };
    });
}
}

```

```

// Add external service check
addExternalServiceCheck(serviceName, url, options = {}) {
  this.addCheck(`external_${serviceName}`, async () => {
    const start = Date.now();
    const response = await fetch(url, {
      method: options.method || 'GET',
      timeout: options.timeout || 5000,
      headers: options.headers || {}
    });

    const latency = Date.now() - start;

    if (!response.ok) {
      throw new Error(`Service responded with status ${response.status}`);
    }

    return {
      status: latency > 2000 ? 'warning' : 'healthy',
      details: {
        url,
        status: response.status,
        latency: `${latency}ms`
      }
    };
  }, { critical: false });
}

// Run single health check
async runCheck(checkName) {
  const checkConfig = this.checks.get(checkName);
  if (!checkConfig) {
    throw new Error(`Health check '${checkName}' not found`);
  }

  const start = Date.now();

  try {
    const result = await Promise.race([
      checkConfig.check(),
      new Promise((_, reject) =>
        setTimeout(() => reject(new Error('Check timeout')),
checkConfig.timeout)
      )
    ]);

    return {
      name: checkName,
      status: result.status || 'healthy',
      duration: Date.now() - start,

```

```

        timestamp: new Date().toISOString(),
        details: result.details || {},
        critical: checkConfig.critical
    };
} catch (error) {
    return {
        name: checkName,
        status: 'unhealthy',
        duration: Date.now() - start,
        timestamp: new Date().toISOString(),
        error: error.message,
        critical: checkConfig.critical
    };
}
}

// Run all health checks
async runAllChecks() {
    const checkNames = Array.from(this.checks.keys());
    const results = await Promise.allSettled(
        checkNames.map(name => this.runCheck(name))
    );

    const checks = results.map((result, index) => {
        if (result.status === 'fulfilled') {
            return result.value;
        } else {
            return {
                name: checkNames[index],
                status: 'unhealthy',
                error: result.reason?.message || 'Unknown error',
                timestamp: new Date().toISOString(),
                critical: this.checks.get(checkNames[index]).critical
            };
        }
    });

    // Determine overall health
    const criticalChecks = checks.filter(check => check.critical);
    const hasUnhealthyCritical = criticalChecks.some(check => check.status === 'unhealthy');
    const hasWarnings = checks.some(check => check.status === 'warning');

    let overallStatus = 'healthy';
    if (hasUnhealthyCritical) {
        overallStatus = 'unhealthy';
    } else if (hasWarnings) {
        overallStatus = 'warning';
    }
}

```

```

return {
  service: this.serviceName,
  status: overallStatus,
  timestamp: new Date().toISOString(),
  uptime: process.uptime(),
  version: process.env.npm_package_version || '1.0.0',
  checks: checks.reduce((acc, check) => {
    acc[check.name] = check;
    return acc;
  }, {})
};
}

// Get health summary
async getHealthSummary() {
  const fullHealth = await this.runAllChecks();

  return {
    service: fullHealth.service,
    status: fullHealth.status,
    timestamp: fullHealth.timestamp,
    uptime: fullHealth.uptime,
    checksCount: {
      total: Object.keys(fullHealth.checks).length,
      healthy: Object.values(fullHealth.checks).filter(c => c.status ===
'healthy').length,
      warning: Object.values(fullHealth.checks).filter(c => c.status ===
'warning').length,
      unhealthy: Object.values(fullHealth.checks).filter(c => c.status ===
'unhealthy').length
    }
  };
}
}

module.exports = HealthChecker;

```



Chapter 4 Summary

Congratulations! You've mastered **enterprise-level microservices architecture**:



Architecture Mastery

- **Microservices decomposition** from monolithic applications

- **API Gateway** with intelligent routing and load balancing
- **Service registry** and dynamic discovery
- **Health monitoring** across distributed systems



Asynchronous Processing

- **Message queues** with Bull and Redis
- **Event-driven communication** between services
- **Job processing** with retry logic and error handling
- **Background task management** and scheduling



Event-Driven Design

- **Event bus** implementation with Redis pub/sub
- **Cross-service communication** without tight coupling
- **Event sourcing** patterns for audit trails
- **Saga patterns** for distributed transactions



Data Architecture

- **Database per service** pattern
- **Data consistency** strategies
- **Cross-service data synchronization**
- **CQRS (Command Query Responsibility Segregation)**



Enterprise Monitoring

- **Distributed health checks** across services
- **Performance monitoring** and alerting
- **Service dependency tracking**
- **Failure detection** and recovery



Scalability Features

- **Horizontal scaling** strategies
- **Load balancing** algorithms
- **Circuit breaker** patterns
- **Graceful degradation** under load



What You Can Build Now

With 4 chapters completed, you can architect and build:

Enterprise Applications

- **Multi-tenant SaaS platforms**
- **High-traffic social media networks**
- **E-commerce marketplaces**
- **Real-time collaborative tools**
- **IoT data processing platforms**

Scalable Systems

- Applications handling **millions of requests/day**
- **Distributed systems** across multiple servers
- **Event-driven architectures** with complex workflows
- **Microservices** that can scale independently

Production Infrastructure

- **Container orchestration** with Docker/Kubernetes
- **CI/CD pipelines** for automated deployment
- **Monitoring and alerting** systems
- **High-availability** setups with failover

Remaining Chapters Preview

Chapter 5: AI/ML Integration & Advanced APIs

- GraphQL implementation
- AI/ML model integration
- Advanced search with Elasticsearch
- Third-party service integrations

Chapter 6: Cloud Native & Enterprise

- AWS/Azure/GCP deployment
- Kubernetes orchestration
- Advanced security patterns
- Enterprise integration patterns

Ready for Chapter 5?

You're now at an **expert level** and can build enterprise-grade applications! The next chapter will add cutting-edge features like AI integration and advanced APIs.

Want to continue to Chapter 5: AI/ML Integration & Advanced APIs? 🤖